

RETO 2 — PATRONES DE DISEÑO ARQUITECTÓNICO

Curso: Arquitectura de Software | Modalidad: equipos de máximo 4 integrantes | Porcentaje: 20 %

Fecha de Entrega del vídeo y documento: **Domingo 6 de septiembre, 23:59:59**

1. Correo de la Líder Técnica (Contextualización)

De: Líder Técnica, Dirección de Ingeniería.

Para: los arquitectos junior del equipo de la dirección de sistemas.

Asunto: Holissss; un nuevo proyecto.



Recibí lo que entregaron el trimestre pasado y lo llevé al comité. Sirvió: por primera vez pude explicar con evidencia por qué este sistema era caro de cambiar, y aprobaron presupuesto para seguir. Eso se los reconozco.

Ahora les voy a decir lo que el comité no vio y yo sí. SOLID les dio un sistema correcto. No me dio un sistema robusto. Son cosas distintas y el equipo de soporte ya me lo está cobrando.

Hoy tenemos responsabilidades separadas y dependencias invertidas, pero cada vez que aparece un tipo nuevo de algo, alguien tiene que ir a modificar el sitio donde se arman los objetos. La decisión de qué implementación usar quedó regada en condicionales que crecen. Tenemos interfaces limpias y bien pensadas, y ningún lugar donde se pueda leer cómo colaboran entre ellas. Cuando alguien pregunta cómo se ensambla el sistema, la respuesta honesta es: hay que leerlo todo.

Eso no es un fracaso de lo que hicieron. Es la siguiente capa del problema. Separar responsabilidades es condición necesaria para que un sistema evolucione, pero no basta: falta resolver cómo se construyen las cosas, cómo se componen entre sí y cómo se toman las decisiones de comportamiento en tiempo de ejecución. Para eso existen los patrones, y para eso los quiero.

Les pongo tres límites y no voy a negociar los.

Primero: el comportamiento observable sigue congelado. Ni una salida, ni un cálculo, ni una regla. Solo cambian las solicitudes que yo autorice.

Segundo: no me cambien el estilo arquitectónico. No quiero oír «clean», ni «hexagonal», ni «lo partimos en servicios», ni «le metemos un framework». Si me traen eso, entiendo que no leyeron el encargo. El trabajo de este trimestre está en cómo colaboran los objetos dentro del back que ya tenemos. Ese nivel, y ninguno más arriba.

Tercero: SOLID no se toca. No acepto un patrón que compre robustez rompiendo un principio que ya pagamos. Si al aplicar algo un principio queda en riesgo, quiero verlo declarado y quiero ver cómo lo compensan. Un patrón mal aplicado deshace SOLID con una elegancia que asusta, y ya vi ese error en equipos con más experiencia que ustedes.

Sobre la IA: el trimestre pasado les pedí criterio. Este trimestre se los voy a medir con una fórmula, y les voy a auditar registros al azar durante la sustentación. Se los digo de frente para que no haya sorpresa. La herramienta les va a nombrar el patrón correcto en quince segundos; eso ya no vale nada. Lo que vale es que ustedes puedan decirme qué evidencia del código justifica ese patrón, qué otras dos opciones evaluaron, qué les cuesta la que escogieron y en qué caso habrían decidido distinto. Si aceptan todo lo que la herramienta les dice, es dependencia. Si rechazan todo para verse críticos, es teatro. Las dos cosas las voy a penalizar.

2. Qué se va a evaluar

Este reto NO evalúa cuántos patrones conoce el equipo. Evalúa cómo decide y cómo justifica la incorporación de estos para un nuevo producto. Concretamente:

- Las decisiones de diseño deben ser del equipo. Pueden apoyarse en IA y deben hacerlo, pero tienen que poder defender cada decisión con evidencia del código propio, no con lo que dijo la herramienta.
- El TO-BE diseñado documentará con precisión qué estructura sale (estructuras rojas), qué estructura entra, cómo se relacionan entre sí y qué impacto tiene el cambio. Para ello, será muy bien valorado que usen un diagramador que les permita hacer diseños por capas. Una capa tendrá el AS IS (diseño con SOLID) y otra capa super puesta, el TOBE (diseño nuevo con SOLID + Patrones)
- Van a demostrar, no solo afirmar, que el diseño sigue cumpliendo SOLID después de introducir los patrones.
- Presentarán un análisis de riesgo real de incorporar esos patrones y un plan concreto para hacer el cambio.
- Desarrollarán dos vistas que les permitirá explicar su arquitectura a dos audiencias distintas: al negocio y al equipo de desarrollo.

PILAS :

Todo patrón que agreguen al diseño TO BE, debe estar anclado a un punto concreto del diseño AS IS que puede ser robustecido para optimizarlo.

Un patrón adoptado solo porque es buena práctica, porque la herramienta lo sugirió o porque simplemente, les pareció bonito, es sobre-ingeniería, y se penaliza igual que la rigidez que pretendía resolver.

3. Alcance

Sí se interviene	No se interviene (y descuenta)
Cómo se crean los objetos y dónde se decide qué implementación concreta se instancia.	Migrar a arquitectura limpia, hexagonal, por capas o a servicios separados. Deben enfocarse en fortalecer el dominio del negocio.
Cómo se componen y relacionan las estructuras existentes.	Frameworks, contenedores de inyección automática, ORM o librerías que resuelvan el problema por ustedes.
Cómo se selecciona y coordina el comportamiento en tiempo de ejecución.	Base de datos real, red, nube, concurrencia o interfaz gráfica.
El punto donde se ensambla el sistema.	Reescribir el sistema o cambiar de lenguaje.

El trabajo está en cómo colaboran los elementos dentro del back que ya tienen. Ese nivel y ninguno más arriba. Un entregable que resuelva el problema cambiando el estilo arquitectónico se considera fuera de encargo.

4. El equipo

Cada integrante responde formalmente por un frente y lo sustenta en el video. Todos deben conocer el conjunto.

Rol	Responde por
Arquitecto Líder	Detección de los puntos rígidos, selección y descarte de patrones, diseño del TO-BE.
Arquitecto de Verificación	Demostración de que SOLID sigue en pie y de que el comportamiento no cambió.
Arquitecto de Riesgos y despliegue	Análisis de riesgo y plan de cambio.

Rol	Responde por
Arquitecto de comunicación gráfica (vistas)	Las dos vistas, la bitácora de decisiones y el armado del documento.

5. Actividades

Seis actividades. Cada una produce un artefacto que va al documento de sustentación.

Actividad 1 — Encontrar los puntos de dolor de la arquitectura actual

Antes de abrir cualquier herramienta, cada integrante relee el código del Reto 1 por su cuenta y anota los puntos donde el diseño, aun siendo correcto, sigue siendo un punto de dolor. o caro de cambiar. Después se consolidan en grupo.

Entregable: Análisis de puntos de dolor.

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad
P-01		Descripción del síntoma, no del principio violado	N clases y M archivos que hay que abrir	Alta / Media / Baja

- Mínimo cinco puntos de dolor, al menos tres encontrados sin IA.
- La columna de costo a van a medir contando clases y archivos reales. No se acepta alto, medio o bajo.
- Al menos uno debe quedar marcado como no se interviene, con el argumento de por qué el remedio cuesta más que el problema.

Actividad 2 — Decidir sobre los patrones a incorporar

Aquí es donde se evalúa el criterio del equipo. Se evalúan varios patrones candidatos y se adoptan pocos.

Entregable 2.1: tabla de decisión de patrones.

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
	Creacional / Estructural / Comportamiento	P-xx		Adoptado / Descartado	

- Mínimo seis patrones evaluados, con al menos dos de cada familia. (siempre se valorará mejor evaluar más)
- Se adoptan entre tres y cinco patrones. Adoptar más exige justificarlo.
- Al menos dos patrones con descartes argumentados con justificaciones técnicas reales. No solamente decir que no se necesita.

Entregable 2.2: bitácora de decisiones frente a la IA.

Es el registro de cómo trabajaron con la herramienta. Mínimo diez decisiones registradas.

ID	Qué consultaron	Qué propuso la herramienta	Qué hicieron	Argumento propio y evidencia
B-01			Aceptamos / Corregimos / Rechazamos / Fue idea nuestra	Referencia a P-xx, a un archivo o a una medición

Actividad 3 — Diseñar el TO-BE

El centro del entregable. Debe quedar claro qué se va, qué llega, cómo se conecta y qué efecto tiene.

Entregable 3.1.: dos diagramas. (o un diagrama con 2 capas, será muy bien valorado)

- Diagrama de lo que sale: el recorte del diseño actual con los elementos que se retiran o cambian de responsabilidad, marcados.
- Diagrama de lo que entra: el TO-BE, indicando en cada clase a qué patrón pertenece y qué papel cumple dentro de él. En negro lo que se conserva sin cambios.

Entregable 3.2.: tabla de cambio estructural.

ID	Elemento	Estado	Qué hacía antes	Qué hace ahora	Quién dependía de él y cómo se reconecta
E-01		Sale / Entra / Se transforma			

Entregable 3.3.: ficha por cada patrón adoptado (máximo una página cada una).

Campo	Contenido
Patrón y punto de dolor que resuelve	Cuál es y a qué P-xx responde, con archivo y línea.
Alternativas que evaluaron	Mínimo dos, y una de ellas debe ser no hacer nada.
Qué sale y qué entra	Elementos que desaparecen y participantes nuevos con su papel.
Cómo se relaciona	Quién construye a estos objetos, quién los usa y si interactúa con otro patrón adoptado.
Impacto	Clases creadas, modificadas y eliminadas. Efecto sobre las solicitudes de cambio del Anexo B.
Qué cuesta	Lo que se paga: más clases, más indirección, más difícil de depurar. Una decisión sin costo declarado no es una decisión.
Origen	Idea propia, o propuesta de la herramienta aceptada, corregida o rechazada. Referencia a la bitácora.

Actividad 4 — Demostrar que SOLID sigue en pie

Un patrón mal aplicado deshace SOLID con mucha elegancia. Hay que demostrar que no pasó.

Entregable 4.1.: matriz de verificación.

Una fila por patrón adoptado, una columna por principio. Valores: Refuerza, Neutro, Tensionado pero compensado, Roto. Toda celda distinta de Neutro (*Ahora no van a poner todas las celdas neutro*) necesita una línea de evidencia debajo. Ninguna celda puede quedar en Roto sin que el equipo lo declare y explique qué beneficio lo compensa.

Patrón adoptado	SRP	OCP	LSP	ISP	DIP

Errores típicos que no deben cometer, por favor revisar:

Situación	Qué principio rompe
Una fábrica que crece con un condicional por cada tipo nuevo.	OCP: movieron el punto de modificación en vez de eliminarlo.
Una fachada que va absorbiendo lógica de negocio.	SRP: termina siendo un objeto que lo hace todo.
Un punto de acceso global de instancia única.	DIP y testabilidad: el consumidor vuelve a depender de algo concreto.
Un método plantilla con pasos que alguna subclase no puede cumplir.	LSP: la subclase implementa el paso vacío y deja de ser sustituible.
Una envoltura que cambia el contrato de lo que envuelve.	LSP: el cliente nota la diferencia.

Entregable 4.2.: Evidencias de que el comportamiento no cambia.

- Como se comportan los casos del Reto 1 con los nuevos patrones y al menos cuatro nuevos que recorran lo que los patrones tocan.
- Salidas del sistema antes y después, lado a lado, demostrando que coinciden.

Actividad 5 — Análisis de Riesgos

Un arquitecto líder presenta al negocio además del diseño, un análisis de los riesgos que se deben mitigar para los cambios que suponen la implementación del TO BE.

Entregable 5.1.: registro de riesgos. Mínimo 3.

ID	Riesgo (si ocurre X, entonces Y)	Prob	Imp	Exp	Qué hacen para evitarlo	Cómo se enteran de que está pasando (el riesgo se está materializando)
R-01		1-5	1-5	PxI		Señal observable

Actividad 6 — Dos vistas

El mismo diseño explicado a dos audiencias que no comparten lenguaje. No es un resumen y su versión larga: son dos documentos con propósitos distintos.

Vista para el negocio	
Para quién	La Líder Técnica y quien aprueba el presupuesto. Personas que deciden y asumen el riesgo, y que no leen código.
Qué debe responder	Qué le van a hacer al sistema y qué no cambia. Dónde se está yendo hoy el tiempo y el dinero. Qué gana el negocio, en tiempo de respuesta a una solicitud y en riesgo de romper algo. Qué cuesta. Qué riesgos hay, en lenguaje de operación. Qué necesitan del negocio. Qué pasa si no se hace.
Qué no puede aparecer	Nombres de patrones. Nombres de clases o archivos. Diagramas UML. Siglas sin desarrollar, incluida SOLID. Las palabras refactorizar, desacoplar e inyección de dependencias.
Prueba de que sirve	Preséntenla a alguien ajeno al equipo y sin formación técnica (Evidencia en el vídeo), y anoten qué entendió. Si al quitar las palabras técnicas la vista se queda sin contenido, es que no había contenido de negocio.

Vista para el equipo de desarrollo	
Para quién	El ingeniero que entra al equipo dentro de seis meses y tiene que hacer un cambio sin romper nada. No estuvo en ninguna reunión.
Qué debe responder	Qué patrones hay, dónde vive cada uno y cómo se relacionan entre sí. Dónde se ensambla el sistema. Qué reglas no se deben romper y por qué. Qué deuda quedó pendiente.
El artefacto central	La guía de dónde tocar: una tabla que para cada tipo de cambio previsible diga qué hay que crear, qué hay que modificar y qué no se debe tocar. Mínimo cinco filas, cubriendo las tres solicitudes del Anexo B.
Prueba de que sirve	Alguien que no participó en el diseño debe poder ubicar, solo con este documento, dónde hacer un cambio.

6. Entregables

6.1 Documento de sustentación

Un solo PDF de máximo 15 páginas, paginado y con índice. Es el documento al que vamos a acudir durante la sustentación, así que se juzga por lo rápido que permite encontrar una respuesta, no por su extensión. Debe contener, en el orden, todo lo pedido en el punto **5 Actividades**.

6.2 Código

- El proyecto refactorizado que sigue el diseño del TO BE

6.3 Video de 20 minutos

Participan los cuatro integrantes con cámara. Cada uno presenta su frente. Lo que quede después del minuto 20 no se evalúa.

Tiempo	Contenido	Responsable
0 – 3	Puntos de dolor	Diseño
3 – 6	Vista de negocio, con la evidencia que alguien no técnico lo entiende	Comunicación
6 – 11	El TO-BE: qué sale, qué entra, cómo se relaciona y qué impacto tiene. Recorrido por los dos diagramas y la ficha del patrón más discutido.	Diseño
11 – 14	SOLID sigue en pie: matriz comentada y ejecución en vivo de los doce casos.	Verificación
14 – 17	Riesgos principales con su señal de alerta.	Riesgos y plan
17 – 20	Decisiones frente a la IA: qué les propuso y rechazaron, y por qué. Solicitud de cambio implementada y deuda que dejan declarada.	Comunicación

7. Reglas

- Código que no compila o no ejecuta: el criterio 4 se califica en 0.0.
- Cambio no autorizado en el comportamiento observable: 0.5 sobre la nota final por cada caso.
- Patrón adoptado sin punto rígido que lo justifique: 0.3 sobre la nota final por cada uno.
- Cambio de estilo arquitectónico o uso de framework que resuelva el problema: el criterio 3 no puede superar 2.5.
- Diagramas que no corresponden al código entregado: los criterios 3 y 4 no pueden superar 3.0.
- Sin bitácora de decisiones: el criterio 2 se califica en 0.0.
- Vista de negocio con nombres de patrones, nombres de clases o UML: el criterio 6 no puede superar 3.0.

- Integrante que no participa en el video o no responde sus preguntas: su nota individual baja al menos 1.5.
- Entrega tardía: -0.5 por cada hora.
- Vídeo sin sonido o de mala calidad: se califica en 0.0

8. Rúbrica

Cada criterio se califica de 0.0 a 5.0. Nota del equipo igual a la suma de cada criterio por su peso, ajustada individualmente según lo que cada integrante demuestre en la sustentación.

Criterio (peso)	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
1. Detección de los puntos de dolor (15 %)	Los puntos son reales y trazables a archivo y a un escenario de cambio. El costo está medido contando clases y archivos. La priorización tiene criterio explícito y el punto no intervenido está bien argumentado.	Puntos correctos en su mayoría, con alguno confundido con un hallazgo del Reto 1 o con costo estimado sin medir.	Puntos genéricos, sin ubicación ni evidencia, o que repiten el diagnóstico del reto anterior.
2. Decisión de patrones y criterio propio frente a la IA (20 %)	Los descartes muestran comprensión real de lo descartado. La bitácora tiene correcciones y rechazos argumentados con evidencia del sistema, y decisiones de origen propio. Los registros escogidos al azar se defienden con solvencia.	Evaluación de candidatos completa y bitácora trazable, con varios argumentos que se quedan en lo general. Alguna decisión no resiste la pregunta directa.	Se enumeran patrones sin evaluarlos, no hay descartes argumentados, o la bitácora es decorativa y el equipo no puede defender sus propias decisiones.
3. Diseño TO-BE: qué sale, qué entra, cómo se relaciona e impacto (20 %)	Los dos diagramas son correctos e indican patrón y papel de cada clase. La tabla de cambio es completa: de cada elemento se sabe qué pasó con él y cómo se reconectó. Las fichas tienen alternativas reales y costo declarado.	Diagramas legibles con imprecisiones de notación. Tabla de cambio con omisiones menores. Fichas completas pero con alternativas tratadas por encima.	El diseño no responde a los puntos detectados, no se distingue lo que sale de lo que entra, o las fichas son descriptivas y sin alternativas.
4. Garantía de SOLID y del comportamiento (15 %)	Matriz completa y con evidencia. Todo principio tensionado está declarado y compensado. Los doce casos pasan y las salidas coinciden con las de antes. El código corresponde al diagrama.	Matriz completa con alguna celda sustentada de forma general. Los casos pasan con cobertura algo limitada de los caminos nuevos.	Sin verificación, con principios rotos sin declararlo, o sin evidencia de que el comportamiento se preservó.
5. Análisis de Riesgos (15 %)	Riesgos escritos como condición y consecuencia, con señal de alerta observable y acción concreta..	Registro completo con algún riesgo vago o señal poco observable.	Riesgos genéricos no vinculados al diseño, sin señales de alerta
6. Las dos vistas y la sustentación (15 %)	Las dos vistas responden a su audiencia y no se parecen entre sí. La de negocio se entiende sin conocimiento técnico. La técnica permite a un tercero ubicar dónde tocar. Los cuatro dominan su frente y el documento permite encontrar cualquier respuesta rápido.	Ambas vistas cumplen, con jerga residual en la de negocio o una guía de dónde tocar que cubre solo lo principal. Participación pareja con alguna respuesta imprecisa.	Una sola vista con dos títulos, vista de negocio llena de términos técnicos, o vista técnica que no permite ubicar dónde hacer un cambio. Participación desigual o documento que no sirve para sustentar.

Anexo A — Patrones disponibles

Familia	Patrones
Creacionales	Factory Method, Abstract Factory, Builder, Prototype, Singleton.
Estructurales	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy.
De comportamiento	Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor.

Advertencias sobre los tres que más se aplican mal:

- Singleton: se permite, pero tienen que explicar cómo lo sustituyen en una prueba y qué lo diferencia de una variable global.
- Facade: tienen que declarar su límite, porque tiende a absorber lógica de negocio hasta romper SRP.
- Abstract Factory: si solo tienen una familia de productos, la abstracción adicional probablemente no se justifica.

Anexo B — Solicitudes de cambio

Son las mismas del Reto 1. Se implementa una, distinta de la que ya hicieron.

	Hacienda
SC-1	La hacienda va a comenzar a vender productos derivados del ganado: lácteos, carne, piel.
SC-2	La hacienda necesita conectar a las reses chips para geolocalización.
SC-3	Además de las vacunas, se requerirá llevar la historia clínica de cada res.

	Farmacia
SC-1	La farmacia venderá, además de productos farmacéuticos, cosméticos y productos comestibles: gaseosas, agua, helados y snacks.
SC-2	La farmacia venderá también servicios: inyectología, cambio de vendajes, curaciones básicas.
SC-3	La farmacia manejará convenios con entidades para descuentos y crédito descontable: empresas, bancos, cooperativas, universidades y colegios.